CS3490: Homework 5

Tree recursion

1. Objectives

In this homework, you will generalize the skills you have learned for programming with lists using pattern-matching and recursion, to different kinds of trees. You should strive to master the following topics by the time you are done with this assignment.

1. Understand the basic logic of programming recursive tree functions.
2. Know how some generic higher-order functions, like `map` and `find`, generalize from lists to other datatypes.
3. Be aware of the fact that every recursive datatype has a corresponding `fold` function, which captures the generic pattern behind all functions defined on that datatype by *structural recursion*.

   A function is structurally recursive if one of the arguments to that function comes from a recursive datatype like lists or trees, and every time the function calls itself recursively, the argument given at that input is a subterm/substructure of the original input given to the function.

   In such functions, the input argument is "getting smaller" with each recursive call. A structurally recursive function is thus *guaranteed* to terminate at every input!
4. Knowing how to convert a structurally recursive function of a datatype to a one-line definition using the fold for that datatype.

   For this, it will be crucial to understand the type of the `fold` for that datatype. Basically, each input to the `fold` function corresponds to a unique data constructor for that datatype.

2. Two kinds of trees

The Haskell code below defines two new datatypes of trees.

Each of these tree types can carry data of arbitrary type `a`.

The datatypes differ in what kinds of nodes are allowed, and where in the tree data can be stored.

In both types of trees, the leaves are *not* constants, but carry data and can thus depend on the value of the underlying type. In pattern matching, such leaves must be matched together with a variable which will be of type `a`. So the "base case" for such functions does not look like `[]` or `Nothing`, but is very similar to matching `Left` or `Just`.

The underlying tree types are as follows:

- The type `LTree` has one leaf and one binary node constructor. The data values occur in both.

```
data LTree a = LLeaf a | LNode a (LTree a) (LTree a)
  deriving (Eq,Show)
```

- The type `MTree` has one leaf, one unary node constructor, and one binary node constructor. *The data occurs in leaves and unary nodes only.*

```
data MTree a = MLeaf a | UNode a (MTree a) | BNode (MTree a) (MTree a)
  deriving (Eq,Show)
```

You should start your homework file with the two datatype definitions above.
You will now define functions that do generic tasks for each of these tree types.

3. Recursive programming with trees

For each `X` $\in$ {`L`, `M`}, implement the following functions using pattern-matching and recursion. You should replace every occurrence of `X` with `L`, and then repeat the same problem after replacing `X` with `M`.

1. `getXLeaves :: XTree a -> [a]`
   takes a labeled tree and returns the contents of the *leaves* of the tree in a single list:

2. `maxXDepth :: XTree a -> Integer` returns the depth/height of the `XTree`. This is the same as the length of the longest path from the root to the leaf. If the given tree is a leaf, its height should be 0.

3. `maxXTree :: XTree Integer -> Integer` returns the biggest value occurring in the tree.

4. `uncoveredLeafX :: Integer -> XTree Integer -> Bool`
   returns `True` if the first argument occurs in some leaf of the tree given in the second argument, *but does not occur in any node between that leaf and the root.*
   It returns `False` if every occurrence in the leaves is "covered" by another occurrence higher in the tree (in a node that the leaf is a descendant of).

5. `mapXTree :: (a -> b) -> XTree a -> XTree b` applies a given function to every element in the `XTree`.

6. `applyXfun :: XTree Integer -> XTree Integer` applies the following function to every data element in an `XTree` of integers.

$$f(x) = 2^{x^2} - x$$

(*Hint.* Use `mapXTree` with the right lambda function.)

7. `findXTree :: (a -> Bool) -> XTree a -> Maybe a` searches the tree for an element satisfying the given predicate.

   You may use the following helper function, which combines two `Maybe` values into one, return `Nothing` only if both inputs are `Nothing`.

   ```
   orMaybes :: Maybe a -> Maybe a -> Maybe a
   orMaybes Nothing y = y
   orMaybes x       _ = x
   ```

8. Use `findXTree` to define a function `findXpali :: XTree String -> Maybe String` to search for a palindrome in a given `XTree` of strings.

9. For this problem and the next one,, you should use the functions `foldXTree`, which will now be defined. Pay close attention to the types and the operation of these functions, especially how the inputs to `foldXTree` relate to the datatype constructors in the definition of the `XTree` type.

   ```
   foldLTree :: (a -> b -> b -> b) -> (a -> b) -> LTree a -> b
   foldLTree n l (LLeaf x)         = l x
   foldLTree n l (LNode x t1 t2)   = n x (foldLTree n l t1) (foldLTree n l t2)

   foldMTree :: (b -> b -> b) -> (a -> b -> b) -> (a -> b) -> MTree a -> b
   foldMTree n u l (MLeaf x)       = l x
   foldMTree n u l (UNode x t)     = u x (foldMTree n u l t)
   foldMTree n u l (BNode t1 t2)   = n (foldMTree n u l t1) (foldMTree n u l t2)
   ```

   Use `foldXTree` to redefine `getXLeaves` as `getXLeaves'` that avoids pattern-matching its input, and does not call itself recursively.

10. Use `foldXTree` to redefine `uncoveredLeafX` as `uncoveredLeafX'` that avoids pattern-matching its input, and does not call itself recursively.

4. Sample output

Add the following definitions to your Haskell file.

```
exLTree :: LTree Integer
exLTree = LNode 5 (LLeaf 4)
                  (LNode 3 (LNode 2 (LLeaf 5) (LLeaf 1))
                           (LLeaf 7))
exMTree :: MTree Integer
exMTree = BNode (UNode 5 (BNode (MLeaf 1) (MLeaf 10)))
                (BNode (UNode 3 (MLeaf 3)) (UNode 4 (MLeaf 3)))
```

The following lists some examples of above functions being tested on the two trees above.

```
ghci> getLLeaves exLTree
[4,5,1,7]
ghci> getMLeaves exMTree
[1,10,3,3]
ghci> maxLDepth exLTree
3
ghci> maxMDepth exMTree
3
ghci> maxLTree exLTree
7
ghci> maxMTree exMTree
10
ghci> uncoveredLeafL 4 exLTree
True
ghci> uncoveredLeafL 5 exLTree
False
ghci> uncoveredLeafL 6 exLTree
False
ghci> uncoveredLeafM 3 exMTree
True
ghci> uncoveredLeafM 4 exMTree
False
ghci> mapLTree (*2) exLTree
LNode 10 (LLeaf 8) (LNode 6 (LNode 4 (LLeaf 10) (LLeaf 2)) (LLeaf 14))
ghci> mapMTree (*2) exMTree
BNode (UNode 10 (BNode (MLeaf 2) (MLeaf 20))) (BNode (UNode 6 (MLeaf 6))
(UNode 8 (MLeaf 6)))
ghci> applyLfun exLTree
LNode 33554427 (LLeaf 65532) (LNode 509 (LNode 14 (LLeaf 33554427) (LLeaf 1))
(LLeaf 562949953421305))
ghci> applyMfun exMTree
   BNode (UNode 33554427 (BNode (MLeaf 1) (MLeaf 126765060022822940149670320536 6)))
   (BNode (UNode 509 (MLeaf 509)) (UNode 65532 (MLeaf 509)))
ghci> findLTree (> 5) exLTree
Just 7
ghci> findLTree even exLTree
Just 4
ghci> findLTree (<= 0) exLTree
Nothing
ghci> findMTree (\x -> x^2 >= 100) exMTree
Just 10
```

```
ghci> findMTree (== 7) exMTree
Nothing
ghci> findLpali (mapLTree show exLTree)
Just "5"
ghci> findMpali (mapMTree show exMTree)
Just "5"
ghci> findLpali (mapLTree show $ applyLfun exLTree)
Just "1"
ghci> findMpali (mapMTree show $ applyMfun exMTree)
Just "1"
ghci> getLLeaves' exLTree
[4,5,1,7]
ghci> getLLeaves' (mapLTree (+1) exLTree)
[5,6,2,8]
ghci> getMLeaves' exMTree
[1,10,3,3]
ghci> getMLeaves' (mapMTree (*2) exMTree)
[2,20,6,6]
ghci>
```